마무리: 동작하는 AI에서 통제 가능한 AI 애플리케이션으로

동작 여부를 넘어, 무엇을 관찰하고 어디에서 통제할 것인가?

4대 핵심 축 회고: 우리가 설계한 네 가지 통제 지점

흐름(FLOW CONTROL) 관찰 · Routing

어떤 경로로 처리되는가?

trace · span · event로 내부 흐름을 관찰하고, Incident 특성에 따라 QUICK(결정론), STANDARD(일반), CRITICAL(정책검증)로 경로를 분기

통제: Micrometer Tracing, Langfuse 관찰, 결정론적 Routing Workflow

비용(COST CONTROL) 선택 · 제한

얼마만큼의 Context를 허용하는가?

관련 데이터가 축적되더라도 모두 보내지 않고, Runbook 중복 제거와 Top N, 외부 입력 및 이전 출력 재주입에 명시적인 상한 적용

통제: Selection Budget (Top N), Size Budget (Hard Limit), Input Token 45%+ 절감

품질(QUALITY CONTROL) 검증 · 개선

구조와 내용 품질이 충분한가?

Java record 매핑 성공 뒤에도 필수값·개수·순위의 기본 계약을 검증하고, Evaluator–Optimizer 루프로 근거 일치성 및 구체성을 최대 1회 개선

통제: Validator Gate, 1회 품질 재시도, Java Fallback, Evaluator-Optimizer

위험(RISK CONTROL) 분류 · 정책 적용

이 추천을 그대로 노출해도 되는가?

외부 severity를 검증하여 실제 처리값을 확정하고, 추천 Action의 위험도를 분류하여 안전한 조치만 필터링한 뒤 구조화된 응답으로 전달

통제: minimumSeverity 정책, Action 위험 분류, Advisor로 공통 정책 컨텍스트 주입

설계 관점: 네 가지 축은 순차적으로 실행되는 개별 단계가 아니라, **요청 흐름 전반에서 서로 연결되어 작용하는 통제 관점**입니다.

아키텍처 조망: Incident Responder — Final Control Map

CROSS-CUTTING **Observability:** trace / span / event · Micrometer Tracing · Langfuse Trace Tree 전송 (요청 전 구간에 걸친 횡단 관찰)

[ENTRY & ROUTING] **Incident Request** → IncidentRouter (severity · system · symptoms 평가) QUICK vs STANDARD / CRITICAL 분기

분기 A: QUICK_TRIAGE 결정론적 Triage 응답

낮은 심각도의 지연 증상에 대해 모델을 호출하지 않고 즉시 안전한 정적 결과 반환

LLM 0회 호출 (비용·지연 제로)

분기 B: IncidentResponseWorkflow Context Budget: Runbook/History Top N & 크기 상한

1. **IncidentAnalysisService** (장애 분석 생성) 내부: QualityControlService (계약검증 + Evaluator-Optimizer)
2. **IncidentActionRecommendationService** (대응 액션 추천 생성) 내부: QualityControlService (계약검증 + Evaluator-Optimizer)
3. **IncidentRiskPolicyService** (위험 분류 및 Java 정책 적용) ActionRiskClassifier + 최종 안전 응답 (0~3개)

CROSS-CUTTING **Policy Advisor:** IncidentPolicyAdvisor → 공통 EscalationDecision 컨텍스트를 모든 관련 LLM 호출 Prompt에 일관 주입 (호출 경계의 정책 일관성 보장)

앞으로의 설계 나침반: 앞으로 AI 애플리케이션을 설계할 때

01

책임 구분 — AI의 판단과 Java의 규칙을 구분한다

LLM은 분석과 액션 추천을 생성하고, 어떤 출력을 채택하고 언제 멈출지는 애플리케이션의 결정론적 규칙으로 통제합니다.

`Authority Decoupling`

02

통제 지점 설계 — 관찰·검증·제한·정책 적용 지점을 명확히 한다

Flow(흐름 분기), Cost(예산 상한), Quality(계약 검증 및 품질 루프), Risk(위험 분류 및 정책 통제)를 설계의 기본 축으로 삼습니다.

`4-Axes Governance`

03

필요에 따른 선택 — 패턴은 실제 문제가 요구할 때 도입한다

복잡한 멀티 에이전트 패턴을 맹목적으로 추종하지 않고, 기존 구조의 확장 가능성을 먼저 검토하며 실질적 필요에 따라 선택합니다.

`Pragmatic Adoption`

> “AI가 항상 올바르게 판단하도록 만드는 것이 아니라,
> **AI의 판단을 애플리케이션이 검증하고 통제할 수 있도록 설계합니다.**”